

I2S Audio Playback Experiment

I2S Audio Playback Experiment

- 1. Audio Playback Overview
- 2. Core Concepts
 - 2.1 I2S Audio Bus
 - 2.2 ES8311 Codec
 - 2.3 WAV File Format
 - 2.4 SPIFFS Flash Filesystem
 - 2.5 WS2812 RGB LED
- 3. ESP32-S3 Audio Hardware Features
- 4. Project Architecture and Flow
 - 4.1 Playback Data Flow
- 5. Hardware Dependencies
- 6. Source Code Details
 - 6.1 app_speech.h — Macro Configuration and Interfaces
 - 6.2 app_speech.c — Audio Subsystem Initialization
 - 6.3 wav_file_player.c — WAV Player
 - 6.4 rgb_led.c — WS2812 RGB LED
 - 6.5 main.c — Main Entry and Button State Machine
- 7. Operation Flow
 - 7.1 menuconfig Configuration
 - 7.2 Build and Flash
 - 7.3 Normal Playback
 - 7.4 Replacing Playback Files
 - 7.5 Edge Cases
- 8. Common Issues

1. Audio Playback Overview

WAV audio playback is one of the most fundamental and important functions in embedded audio systems. Mastering core technologies such as I2S audio bus, audio codec driver, WAV file parsing, and DMA transfer is a prerequisite for developing products like smart speakers, voice assistants, alarms, and voice broadcast systems. In IoT devices, audio playback is widely used in scenarios such as **voice prompts, music playback, device alerts, and human-machine interaction voice feedback.**

This project implements a WAV audio player based on the BOOT button, driving the ES8311 audio codec (DAC mode) via the I2S bus, reading mono WAV files from the SPIFFS flash filesystem, duplicating to stereo output through the I2S TX channel, and driving a speaker via the NS4150 power amplifier.

Core capabilities:

Function	Description
File Storage	SPIFFS flash filesystem, stores WAV audio files
Audio Decoding	ES8311 DAC, 16-bit / 16kHz / stereo output

Function	Description
Playback Control	BOOT button (GPIO0): short press to play/pause, long press to stop
LED Feedback	WS2812 RGB LED: rainbow hue animation during playback, off when not playing
Amplifier Drive	NS4150 Class D amplifier, GPIO7 enable

2. Core Concepts

2.1 I2S Audio Bus

I2S (Inter-IC Sound) is the standard digital audio transmission protocol, containing the following signal lines:

Signal	Full Name	Function
MCLK	Master Clock	Master clock (codec working clock), = sample rate x 256
BCLK	Bit Clock	Bit clock, one pulse per bit of data
WS / LRCLK	Word Select	Frame clock, low = left channel, high = right channel
DOUT	Data Out	Transmit data (ESP32 -> DAC)
DIN	Data In	Receive data (ADC -> ESP32), this project uses DOUT only

I2S uses Philips standard format (left-aligned, MSB first), WS toggles one BCLK before the data MSB.

2.2 ES8311 Codec

The ES8311 is a low-power audio codec from Shunli Semiconductor, configured via I2C registers with audio data transmitted over I2S:

Feature	Parameter
I2C Address	0x18 (7-bit)
DAC Resolution	24-bit
ADC Resolution	24-bit
Sample Rate	8~96 kHz
Working Mode	DAC / ADC / BOTH

This project uses DAC mode, operating the ES8311 through the I2C control interface and I2S data interface封装 provided by the `esp_codec_dev` library.

2.3 WAV File Format

WAV is Microsoft's waveform audio file format. In PCM encoding, it consists of **44-byte header + raw PCM data**:

Offset	Size	Field	Description
0x00	4	"RIFF"	Magic number
0x04	4	file_len	File size - 8
0x08	4	"WAVE"	Format identifier
0x0C	4	"fmt "	Format chunk identifier
0x10	4	16	PCM format chunk size
0x14	2	1	Audio format (1=PCM)
0x16	2	channels	Number of channels (1=mono, 2=stereo)
0x18	4	sample_rate	Sample rate (e.g., 16000 Hz)
0x1C	4	byte_rate	Byte rate (sample rate x channels x bit depth/8)
0x20	2	block_align	Frame alignment (channels x bit depth/8)
0x22	2	bit_depth	Bit depth (16)
0x24	4	"data"	Data chunk identifier
0x28	4	data_len	PCM data byte count

This project's WAV files are 16kHz / 16-bit / mono, duplicated to stereo output during playback.

2.4 SPIFFS Flash Filesystem

SPIFFS is a SPI Flash filesystem built into ESP-IDF, suitable for storing read-only audio resources:

Feature	Description
Storage Medium	On-chip SPI Flash
Partition Type	data, spiffs
Mount Path	/spiffs
File Operations	Standard POSIX fopen/fread/fclose

2.5 WS2812 RGB LED

The WS2812 is a single-wire cascaded RGB LED, driven by the RMT (Remote Control) peripheral with precise timing:

Timing Parameter	Value	Description
T0H	0.40 us	0 code high level
T0L	0.85 us	0 code low level
T1H	0.80 us	1 code high level
T1L	0.45 us	1 code low level
RES	>50 us	Reset latch signal

Timing Parameter	Value	Description
Data Format	GRB byte order	24-bit color

This project uses the RMT peripheral for direct WS2812 driving, instead of the `espressif/led_strip` third-party component. RMT is a built-in IDF peripheral driver (`driver/rmt_tx.h`), no additional dependencies needed in `idf_component.yml`. Using 80MHz resolution to encode GRB timing pulses, the code is more low-level but requires no additional components. Comparison of the two approaches:

Approach	Dependencies	Advantages	Disadvantages
This project: RMT direct drive	None (IDF built-in)	Zero extra dependencies, fine-grained timing control	Requires manual encoding logic
Alternative: led_strip component	<code>espressif/led_strip</code>	Simple API, ready to use	Additional component dependency

For information about the `led_strip` component usage, refer to 2.Basic Peripherals->10.RGB LED Effects.

3. ESP32-S3 Audio Hardware Features

Feature	Parameter
I2S Peripherals	2 (I2S0 / I2S1)
I2C Peripherals	2 (I2C0 / I2C1)
RMT Peripherals	4 channels, supports 80MHz resolution
SPI Flash	Up to 16MB
GPIO Matrix	Any GPIO mapped to any peripheral

Pin assignment: This project uses I2S0 for I2S, I2C0 for I2C, and RMT bound to GPIO48.

4. Project Architecture and Flow

```
app_main()
|- Codec_I2S_init()    <- Initialize audio playback chain
|  |- i2c_init()       <- Create I2C host bus, scan devices
|  |- i2s_driver_init() <- Create I2S TX channel (master/Philips/stereo)
|    |- write mute data to I2S <- Output silence during pin floating to suppress noise
|    |- ES8311 dummy read stabilization <- Multiple I2C probes after power-on for stabilization
|      |- Init_codec_official() <- Configure ES8311 DAC, open device, set volume to 90%
|      |- Flush mute data to I2S <- Flush pipeline again after codec configuration
```

```

|   |- power_on_codec()    <- Finally enable amplifier (GPIO7), at this point I2S is
already outputting silence
|
|- wav_player_init()      <- Mount SPIFFS and list files
|
|- btn_play_init()        <- Configure BOOT button as polling input (internal pull-up)
|
|- rgb_led_init() / rgb_led_off() <- WS2812 initialization and turn off LED
|
|- while(1) main loop      <- 50Hz button polling + LED effect control
    |- Detect button falling edge (debounce 10ms)
    |- Short press (<800ms):
    |   |- Playing -> Pause/Resume
    |   |- Idle    -> Start playback
    |- Long press (>=800ms): Stop playback
    |- During playback: Rainbow LED step (every 60ms)

```

4.1 Playback Data Flow

```

SPIFFS file -> fopen/fread -> Mono PCM Buffer
    -> Mono->Stereo duplication (write each sample to left and right slots)
    -> i2s_channel_write(tx_handle_)
    -> I2S DOUT -> ES8311 DAC -> NS4150 PA -> Speaker

```

5. Hardware Dependencies

Hardware connections:

Pin	Function	Connection Target
GPIO42	I2C SCL	ES8311 SCL
GPIO41	I2C SDA	ES8311 SDA
GPIO15	I2S MCLK	ES8311 MCLK
GPIO16	I2S BCLK	ES8311 BCLK
GPIO18	I2S WS	ES8311 LRCK
GPIO8	I2S DOUT	ES8311 SDIN (DAC)
GPIO7	PA_EN	NS4150 amplifier enable
GPIO48	WS2812 Data	RGB LED data input
GPIO0	BOOT button	Active-low button

Required components: `esp_codec_dev` / `es8311` / `driver/i2s_std` / `driver/i2c_master` / `driver/rmt_tx` / `esp_spiffs` / `FreeRTOS`

This project uses the following third-party libraries, from the ESP-IDF Component Registry (Component Registry), not IDF built-in. Dependencies must be declared in `main/idf_component.yml` before building, otherwise compilation will fail to find the corresponding headers:

WS2812 LED effects do not use the `espressif/led_strip` third-party component; instead, they use IDF's built-in RMT peripheral for direct drive (see [Section 2.5 WS2812 RGB LED]), so no declaration is needed here.

| `espressif/esp_codec_dev` | ^1.5.10 | Audio codec abstraction layer, encapsulates ES8311 and I2S operations | - |

```
## IDF Component Manager Manifest File
dependencies:
  idf:
    version: '>=4.1.0'
  espressif/esp_codec_dev: ^1.5.10 # Audio codec
```

After declaration, executing `idf.py build` will cause the component manager to automatically download `esp_codec_dev` and its sub-devices (ES8311, ES7210, ES8156, etc.) to the `managed_components/` directory.

6. Source Code Details

Complete source code: `Source Code -> ESP-IDF -> 6.Audio -> Audio_Playing`

The system consists of 4 source files, read in call chain order: `main.c` for startup -> `app_speech.c` provides audio hardware initialization -> `wav_file_player.c` handles file reading and playback tasks -> `rgb_led.c` handles LED status display.

6.1 app_speech.h — Macro Configuration and Interfaces

`app_speech.h` is the header file for the entire audio playback subsystem, defining I2C/I2S pin mappings, audio sampling parameters, codec addresses, and external interface function declarations. All macros are centrally managed for easy unified modification when hardware pins change.

Related pin and parameter macros:

Macro	Value	Description
<code>I2C_SCL_PIN</code> / <code>I2C_SDA_PIN</code>	GPIO42 / GPIO41	I2C pins / I2C clock & data
<code>I2S_MCLK</code> / <code>I2S_BCLK</code> / <code>I2S_WS</code>	GPIO15 / GPIO16 / GPIO18	I2S clock pins / I2S clocks
<code>I2S_DOUT</code>	GPIO8	TX data (to ES8311 DAC) / TX data out
<code>EXAMPLE_VOICE_VOLUME</code>	90	Output volume (0~90) / Output volume
<code>EXAMPLE_SAMPLE_RATE</code>	16000	Sample rate (Hz) / Sample rate
<code>EXAMPLE_MCLK_MULTIPLE</code>	256	MCLK = 16000 x 256 = 4.096MHz

Macro	Value	Description
<code>ES8311_ADDR</code>	0x18	ES8311 7-bit I2C address / I2C address
<code>I2S_NUM</code> / <code>I2C_NUM</code>	I2S_NUM_0 / I2C_NUM_0	Peripheral index
<code>BTN_PLAY_PIN</code>	GPIO0	BOOT button (active-low) / Active-low button
<code>VOICE_EN_PIN</code> / <code>PA_GPIO_NUM</code>	GPIO7	NS4150 amplifier enable pin / PA enable pin

6.2 app_speech.c — Audio Subsystem Initialization

`codec_i2s_init()` initializes the entire playback chain in sequence, **with the amplifier enabled last**, ensuring that when the amplifier is turned on, I2S is already outputting silence to avoid power-on pop noise:

1. `i2c_init()` — I2C host bus

Configures I2C_NUM_0 as host mode, enables internal pull-up, then scans addresses 1~126.

2. `i2s_driver_init()` — I2S TX channel

Configures I2S0 as standard Philips master mode, 16kHz sample rate, 16-bit width, stereo slots, MCLK output 4.096MHz. 6 DMA descriptors, 720 samples per frame, ensuring smooth audio without stuttering. Immediately writes mute data after initialization to avoid noise from floating I2S pins.

```
// Create I2S TX channel (master, standard Philips mode, stereo)
i2s_chan_config_t chan_cfg = {
    .id      = I2S_NUM,
    .role    = I2S_ROLE_MASTER,      /* ESP32 generates BCLK+WS */
    .dma_desc_num = 6,
    .dma_frame_num = 240 * 3,
};
i2s_new_channel(&chan_cfg, &tx_handle_, NULL); /* TX only */

i2s_std_config_t std_cfg = {
    .clk_cfg = {
        .sample_rate_hz = 16000,
        .mclk_multiple = EXAMPLE_MCLK_MULTIPLE, /* 256 -> MCLK=4.096MHz */
    },
    .slot_cfg = {
        .data_bit_width = I2S_DATA_BIT_WIDTH_16BIT,
        .slot_mode      = I2S_SLOT_MODE_STEREO, /* stereo slots */
        .slot_mask      = I2S_STD_SLOT_BOTH,
    },
    .gpio_cfg = {
        .mclk = I2S_MCLK, .bclk = I2S_BCLK,
        .ws   = I2S_WS,   .dout = I2S_DOUT,
    },
};
i2s_channel_init_std_mode(tx_handle_, &std_cfg);
i2s_channel_enable(tx_handle_);
```

3. Init_codec_official() — ES8311 initialization

Creates I2S data interface, I2C control interface, and GPIO interface through the `esp_codec_dev` library封装, to build the ES8311 codec instance. Opens in DAC mode, sets 16kHz / 16-bit / stereo, volume 90%.

```
es8311_codec_cfg_t es8311_cfg = {
    .codec_mode = ESP_CODEC_DEV_WORK_MODE_DAC, /* DAC only */
    .pa_pin     = GPIO_NUM_3,                  /* Library manages PA pin */
    .use_mclk   = true,
};

esp_codec_dev_sample_info_t fs_info = {
    .sample_rate = 16000, /* sample rate */
    .channel     = 2,      /* stereo */
    .bits_per_sample = 16,
};

esp_codec_dev_open(codec_dev_, &fs_info);
esp_codec_dev_set_out_vol(codec_dev_, 90); /* volume limit 90% */
```

4. Mute data flush — Suppress power-on pop noise

After ES8311 configuration, write 320 bytes of mute PCM to I2S again to ensure the codec DAC output is stable at zero level.

5. power_on_codec() — Enable amplifier last

Only after the entire audio chain is ready and I2S is continuously outputting silence, pull GPIO7 high to enable the NS4150 amplifier. This is **the most critical step to eliminate power-on pop noise** — when the amplifier turns on, the speaker receives a silence signal instead of random noise.

```
static void power_on_codec(void)
{
    gpio_reset_pin(PA_GPIO_NUM);
    gpio_set_direction(PA_GPIO_NUM, GPIO_MODE_OUTPUT);
    gpio_set_level(PA_GPIO_NUM, 1);
    vTaskDelay(pdMS_TO_TICKS(100)); /* wait for power stable */
    ESP_LOGI(TAG, "PA enabled GPIO%d", PA_GPIO_NUM);
}
```

6.3 wav_file_player.c — WAV Player

The `wav_file_player` module is responsible for SPIFFS mounting and playback task management.

wav_player_init() — Mount SPIFFS and list files

Mounts the `storage` partition to the `/spiffs` path, automatically formats on first use. After mounting, scans the directory and prints the file list for easy confirmation that WAV files have been correctly burned.

```
void wav_player_init(void)
{
    esp_vfs_spiffs_conf_t conf = {
        .base_path = "/spiffs",
        .partition_label = "storage",
    };
}
```



```

        .max_files                = 5,
        .format_if_mount_failed = true, /* Auto format on first use */
};
esp_vfs_spiffs_register(&conf);

ESP_LOGI(TAG, "SPIFFS mounted");

/* List SPIFFS files for debugging */
DIR *dir = opendir("/spiffs");
if (dir) {
    ESP_LOGI(TAG, "=== SPIFFS files ===");
    struct dirent *ent;
    while ((ent = readdir(dir)) != NULL) {
        ESP_LOGI(TAG, " %s", ent->d_name);
    }
    closedir(dir);
}
}

```

wav_play_task() — Playback task (FreeRTOS Task)

Core playback loop:

1. `fread(&hdr, 44)` reads WAV file header, parses sample rate / bit depth / channel count
2. **Send mute preamble** (~25ms), smooth DAC startup, eliminate startup pop
3. Loop `fread(mono_buf, 256 samples)` reads mono PCM
4. Duplicate each sample to L+R slots: `stereo_buf[i*2] = mono_buf[i]; stereo_buf[i*2+1] = mono_buf[i]`
5. `i2s_channel_write(tx_handle_, stereo_buf, ...)` sends to DAC
6. When paused, poll and wait (`while(s_paused) vTaskDelay`)
7. When stopped, exit loop, send mute frames then close file

Mono PCM data is duplicated to stereo and written to I2S, with each sample value written to both left and right channel slots simultaneously:

```

// Duplicate to left and right channels: {s0,s0, s1,s1, ...}
for (size_t i = 0; i < rd; i++) {
    stereo_buf[i * 2]      = mono_buf[i]; /* Left channel */
    stereo_buf[i * 2 + 1] = mono_buf[i]; /* Right channel */
}
i2s_channel_write(tx_handle_, (uint8_t *)stereo_buf,
                  rd * sizeof(int16_t) * 2, &wd, pdMS_TO_TICKS(1000));

```

Status variables use `volatile bool` because the main loop (`app_main`) and playback task (`wav_play_task`) access them from different FreeRTOS tasks, requiring protection against compiler optimization.

Public interfaces:

Function	Description
<code>wav_player_init()</code>	Mount SPIFFS
<code>wav_player_play_file(path)</code>	Start playback (launch task)
<code>wav_player_pause_resume()</code>	Pause/resume toggle
<code>wav_player_stop()</code>	Stop playback
<code>wav_player_is_busy()</code>	Check if task is alive (playing or paused)
<code>wav_player_is_playing()</code>	Check if currently sending audio
<code>app_speech_play(buf, len, &written)</code>	Write PCM directly to I2S

6.4 rgb_led.c — WS2812 RGB LED

Drives a single WS2812 LED using the RMT peripheral at 80MHz resolution:

- `rgb_led_init()` — Create RMT TX channel and copy encoder
- `rgb_led_set(r, g, b)` — Encode 24 bits in GRB byte order + RES pulse
- `rgb_led_off()` — Set black + wait for transmission complete
- `rgb_led_rainbow_step()` — Rainbow step based on HSV hue wheel

Hue cycles from 0~65535, each step advances approximately 1.4 degrees, 6 sectors cover R->Y->G->C->B->M->R.

The HSV hue wheel is divided into 6 sectors, with linear interpolation within each sector for smooth color transitions:

```
uint8_t sector = (g_hue >> 13) & 0x7; /* Hue sector (0-5) */
uint8_t x      = (g_hue >> 5) & 0xFF; /* Blend factor (0-255) */
switch (sector) {
case 0: r = 255;      gr = x;      b = 0;      break; /* Red->Yellow / R->Y */
case 1: r = 255 - x;  gr = 255;     b = 0;      break; /* Yellow->Green / Y->G */
case 2: r = 0;        gr = 255;     b = x;      break; /* Green->Cyan / G->C */
case 3: r = 0;        gr = 255 - x; b = 255;    break; /* Cyan->Blue / C->B */
case 4: r = x;        gr = 0;        b = 255;    break; /* Blue->Magenta / B->M */
default: r = 255;     gr = 0;        b = 255 - x; break; /* Magenta->Red / M->R */
}
g_hue += 400; /* Advance ~1.4 degrees/step */
```

6.5 main.c — Main Entry and Button State Machine

`app_main()` initializes the audio chain, SPIFFS, button GPIO, and RGB LED in sequence, then enters a 50Hz main loop. The button logic uses **debounce -> falling edge capture -> timing -> rising edge detection** to distinguish between short and long presses, while also supporting **immediate long-press response** (trigger stop when held beyond threshold without waiting for release).

```
// ---- Macros ----
```

```

#define PLAY_FILE        "/spiffs/Love even unto death.wav"  /* Default playback file
*/
#define LONG_PRESS_MS    800          /* Long press threshold (ms) */
#define DEBOUNCE_TICKS  3            /* Debounce tick count */

void app_main(void)
{
    ESP_LOGI(TAG, "===== Audio Player (Button: Play/Pause) =====");

    Codec_I2S_init();    /* I2S+I2C+ES8311 DAC init */
    wav_player_init();   /* SPIFFS mount+file scan */
    btn_play_init();     /* GPIO0 button init (pull-up) */
    rgb_led_init();      /* WS2812 LED init */
    rgb_led_off();       /* Start with LED off */

    ESP_LOGI(TAG, "short press = Play/Pause | Long press = Stop");
    ESP_LOGI(TAG, "=====");

    bool    btn_prev      = true;    /* Previous level (high=released) */
    bool    btn_done      = false;   /* Press already handled */
    uint32_t btn_down_ticks = 0;     /* Press start tick count */
    uint32_t rainbow_tick  = 0;     /* Rainbow step counter */

    while (1) {
        bool btn_raw = gpio_get_level(BTN_PLAY_PIN); /* High=released, low=pressed */

        // Press edge (debounced)
        if (!btn_raw && btn_prev) {
            vTaskDelay(pdMS_TO_TICKS(10));          /* Coarse debounce */
            if (gpio_get_level(BTN_PLAY_PIN) == 0) { /* Confirmed */
                btn_prev      = false;
                btn_done      = false;
                btn_down_ticks = xTaskGetTickCount();
                ESP_LOGI(TAG, "BTN: Pressed");
            }
        }

        // Release edge
        else if (btn_raw && !btn_prev) {
            btn_prev = true;
            if (btn_done) continue; /* Long press already handled */

            uint32_t held = (xTaskGetTickCount() - btn_down_ticks)
                * portTICK_PERIOD_MS;
            if (held < LONG_PRESS_MS) { /* Short press: pause/play */
                if (wav_player_is_busy()) {
                    bool playing = wav_player_pause_resume();
                    ESP_LOGI(TAG, "BTN: %s", playing ? "Resume" : "Pause");
                } else {
                    wav_player_play_file(PLAY_FILE);
                    ESP_LOGI(TAG, "BTN: Play");
                }
            } else { /* Long press: stop */
                wav_player_stop();
            }
        }
    }
}

```

```

        ESP_LOGI(TAG, "BTN: Stop");
    }
}

// Immediate long-press response: stop on threshold
if (!btn_prev && !btn_done) {
    uint32_t held = (xTaskGetTickCount() - btn_down_ticks)
        * portTICK_PERIOD_MS;

    if (held >= LONG_PRESS_MS) {
        btn_done = true;
        wav_player_stop();
        ESP_LOGI(TAG, "BTN: Long -> Stop");
    }
}

// Rainbow LED: ~60ms/step during playback
if (wav_player_is_playing()) {
    if (++rainbow_tick >= 3) {
        rainbow_tick = 0;
        rgb_led_rainbow_step();
    }
} else {
    if (rainbow_tick != 0) rainbow_tick = 0;
    rgb_led_off(); /* Auto off when not playing */
}

vTaskDelay(pdMS_TO_TICKS(20)); /* 50Hz polling */
}
}

```

Button detection flow:

- **Falling edge** (1->0) -> Debounce 10ms -> Confirm press -> Record press time
- **Rising edge** (0->1) -> Skip if long press already handled; if held < 800ms = short press, execute play/pause; if >= 800ms = long press, execute stop
- **Held continuously** >= 800ms -> Immediate long-press response (don't wait for release), directly stop

LED effect logic:

- During playback (`wav_player_is_playing()`) -> Rainbow step every 3 loops (60ms)
- Not playing -> Automatically turn off LED

7. Operation Flow

7.1 menuconfig Configuration

This project depends on a custom partition table (including SPIFFS storage partition) and PSRAM. The following configuration options must be enabled in menuconfig before compilation. For detailed instructions on general settings such as the partition table and Flash size (including menuconfig screenshots), please refer to **Section 4.2.1 in 3.Serial Communication and Storage System->6. USB Emulated Mass Storage Experiment**.

Config Path	Setting	Value	Description
Serial flasher config -> Flash size	Flash size	16 MB	Match onboard 16MB SPI Flash / 16MB SPI Flash
Partition Table -> Partition Table	Custom partition table CSV	partitions.csv	Custom partition (with 10MB SPIFFS storage) / Custom partition with SPIFFS
Component config -> ESP PSRAM	Support for external PSRAM	Enable	Enable 8MB PSRAM / Enable 8MB PSRAM
Component config -> ESP PSRAM	Allow malloc() in external memory	Enable	Allow memory allocation from PSRAM / malloc from PSRAM
Component config -> SPIFFS Configuration	Maximum Number of Partitions	3	Max SPIFFS partitions / Max SPIFFS partitions

The above configuration has been written to the project's `sdkconfig.defaults`. After executing `idf.py set-target esp32s3`, IDF will automatically apply it. If compilation reports an error that a certain function is not enabled, enter `idf.py menuconfig` to check items one by one.

```
# Flash size / Flash size
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y

# Custom partition table / Custom partition table
CONFIG_PARTITION_TABLE_CUSTOM=y
CONFIG_PARTITION_TABLE_CUSTOM_FILENAME="partitions.csv"

# PSRAM / External RAM
CONFIG_SPIRAM=y
CONFIG_SPIRAM_USE_MALLOC=y

# SPIFFS
CONFIG_SPIFFS_MAX_PARTITIONS=3
CONFIG_SPIFFS_CACHE=y
CONFIG_SPIFFS_CACHE_WR=y
CONFIG_SPIFFS_PAGE_CHECK=y
CONFIG_SPIFFS_USE_MAGIC=y
CONFIG_SPIFFS_USE_MAGIC_LENGTH=y
CONFIG_SPIFFS_USE_MTIME=y
```

7.2 Build and Flash

ESP-IDF flashing flow: See `ESP-IDF Development Tutorial` → 1. Before Development → 3. Build and Flash.

7.3 Normal Playback

Step 1: Confirm hardware connections

This development board has ES8311 audio codec chip and NS4150 amplifier onboard, no external modules needed, ready to use after power-on. Ensure the speaker is correctly connected to the amplifier output.

Step 2: Power on and observe serial output:

```
I (1177) MAIN: ===== Audio Player (Button: Play/Pause) =====
I (1187) app_speech: ===== Audio System Init (Playback Only) =====
I (1197) app_speech: Initializing I2C bus...
I (1197) app_speech: I2C initialized (SDA:41, SCL:42)
I (1247) app_speech: ===== I2C Bus Scan =====
I (1247) app_speech:   [FOUND] Device at 0x18
I (1267) app_speech: Found 1 I2C device(s)
I (1267) app_speech: ===== Scan Complete =====
I (1267) app_speech: Initializing I2S driver (TX only)...
I (1377) app_speech: I2S driver initialized (MCLK:15, BCLK:16, WS:18, DOUT:8)
I (1427) app_speech: Initializing ES8311 codec (DAC only)...
I (1427) ES8311: Work in Slave mode
I (1437) I2S_IF: No paired data, current mode: playback
I (1437) I2S_IF: STD: TX, data_bit: 16, slot_bit: 16, ws_width: 16, slot_mode: STEREO,
slot_mask: 0x3
I (1437) I2S_IF: STD: TX, sample_rate_hz: 16000, mclk_multiple: 256, clk_src: 6
I (1467) Adev_Codec: Open codec device OK
I (1467) app_speech: ES8311 codec initialized (sample rate: 16000, volume: 90%)
I (1577) app_speech: PA enabled GPIO7
I (1577) app_speech: =====
I (1577) app_speech:   Audio Player (Loop) Ready!
I (1577) app_speech: =====
I (2407) WAV_PLAYER: SPIFFS mounted
I (2407) WAV_PLAYER: === SPIFFS files ===
I (2407) WAV_PLAYER:   Love even unto death.wav
I (2787) rgb_led: init GPIO48
I (2787) MAIN: Short press = Play/Pause | Long press = Stop
I (2787) MAIN: =====
```

Startup log sequence: Amplifier enable -> I2C bus scan (confirm ES8311 at 0x18) -> I2S driver ready -> ES8311 initialization (16000Hz / 90% volume) -> SPIFFS mount (list WAV files) -> Button prompt.

```

I (1177) MAIN: ===== Audio Player (Button: Play/Pause) =====
I (1187) app_speech: ===== Audio System Init (Playback Only) =====
I (1197) app_speech: Initializing I2C bus...
I (1197) app_speech: I2C initialized (SDA:41, SCL:42)
I (1247) app_speech: ===== I2C Bus Scan =====
I (1247) app_speech: [FOUND] Device at 0x18
I (1267) app_speech: Found 1 I2C device(s)
I (1267) app_speech: ===== Scan Complete =====
I (1267) app_speech: Initializing I2S driver (TX only)...
I (1377) app_speech: I2S driver initialized (MCLK:15, BCLK:16, WS:18, DOUT:8)
I (1427) app_speech: Initializing ES8311 codec (DAC only)...
I (1427) ES8311: Work in Slave mode
I (1437) I2S_IF: No paired data, current mode: playback
I (1437) I2S_IF: STD: TX, data_bit: 16, slot_bit: 16, ws_width: 16, slot_mode: STEREO, slot_mask: 0x3
I (1437) I2S_IF: STD: TX, sample_rate_hz: 16000, mclk_multiple: 256, clk_src: 6
I (1467) Adev_Codec: Open codec device OK
I (1467) app_speech: ES8311 codec initialized (sample rate: 16000, volume: 90%)
I (1577) app_speech: PA enabled GPIO7
I (1577) app_speech: =====
I (1577) app_speech: Audio Player (Loop) Ready!
I (1577) app_speech: =====
I (2407) WAV_PLAYER: SPIFFS mounted
I (2407) WAV_PLAYER: === SPIFFS files ===
I (2407) WAV_PLAYER: Love even unto death.wav
I (2787) rgb_led: init GPIO48
I (2787) MAIN: Short press = Play/Pause | Long press = Stop
I (2787) MAIN: =====

```

Step 3: Short press BOOT button to start playback:

```

I (3200) MAIN: BTN: Pressed
I (3230) MAIN: BTN: Play
I (3230) WAV_PLAYER: WAV: 16000Hz 1ch 16bit, data=12345678B
I (3230) WAV_PLAYER: Playing: /spiffs/Love even unto death.wav

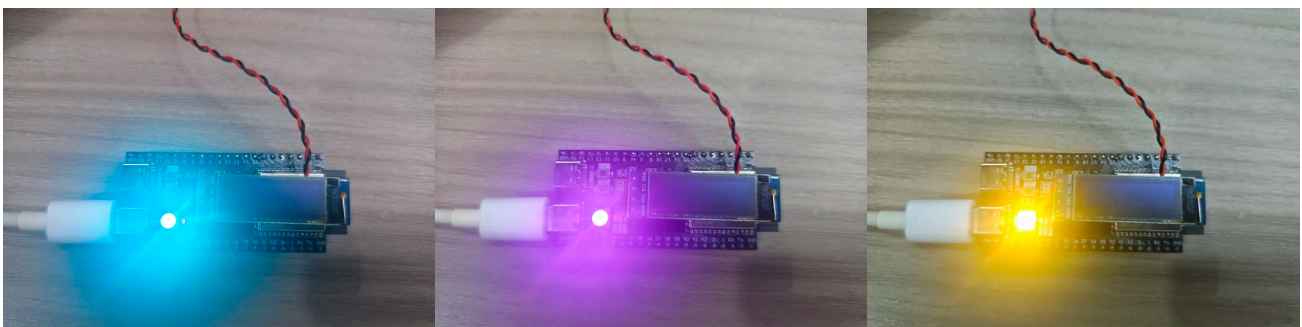
```

LED shows rainbow flowing effect, speaker outputs music. Serial prints WAV format info (sample rate/channels/bit depth/data size).

```

I (348207) MAIN: BTN: Pressed
I (348427) WAV_PLAYER: WAV: 16000Hz 1ch 16bit, data=3974400B
I (348427) WAV_PLAYER: Playing: /spiffs/Love even unto death.wav

```



Step 4: Short press BOOT button to pause playback:

```

I (123000) MAIN: BTN: Pressed
I (123030) MAIN: BTN: Pause

```

Short press again to resume playback, serial prints "Resume".

```
I (418577) MAIN: BTN: Pressed
I (418777) WAV_PLAYER: Paused
```

Step 5: Long press BOOT button (>0.8s) to stop playback:

```
I (421927) WAV_PLAYER: Stop requested
I (421927) MAIN: BTN: Long -> Stop
I (421977) WAV_PLAYER: Done: 3124224 bytes
```

```
I (421927) WAV_PLAYER: Stop requested
I (421927) MAIN: BTN: Long -> Stop
I (421977) WAV_PLAYER: Done: 3124224 bytes
```

LED turns off, playback stops.

Step 6 (Automatic): Playback reaches end of file:

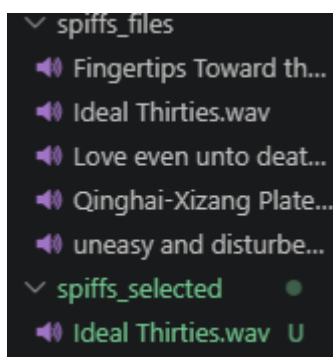
```
I (250000) WAV_PLAYER: Done: 12345678 bytes
```

Auto stop, LED turns off, prints played byte count statistics.

7.4 Replacing Playback Files

The project has two directories related to audio files with different purposes:

Directory	Purpose	Burned
<code>spiffs_files/</code>	Audio archive , stores all prepared WAV files for easy switching	No
<code>spiffs_selected/</code>	Current selected track , copy files from <code>spiffs_files/</code> here to play	Yes, packaged into SPIFFS image



Replacement steps:

1. Prepare new WAV file (**must be 16kHz / 16-bit / mono**), put in `spiffs_files/` archive
2. Copy selected file to `spiffs_selected/` (only this directory's file will be burned to SPIFFS)
3. Open `main.c`, modify `PLAY_FILE` macro so filename **exactly matches** the file in `spiffs_selected/` (including case):

```
#define PLAY_FILE "/spiffs/YourFileName.wav"
```



```
14 // ---- 宏定义 / Macros ----
15 #define PLAY_FILE "/spiffs/Ideal Thirties.wav" // 默认播放文件 / default WAV file
```

4. Rebuild and flash: `idf.py build flash monitor`

If filename doesn't match the macro, serial will print "Failed to open" and playback won't respond.

7.5 Edge Cases

Scenario	Expected Behavior
Short press BOOT during playback	Pause playback, LED turns off; short press again to resume
Long press BOOT during playback (>0.8s)	Stop immediately, send mute frames then close file
Playback reaches end of file	Auto stop, LED turns off, print playback byte count
Short press BOOT when idle	Start playing <code>/spiffs/Love even unto death.wav</code>
Long press BOOT when idle	No effect (no active playback task to stop)
No WAV files in SPIFFS partition	Serial prints "SPIFFS is empty" warning, button has no playback effect
WAV file format mismatch (not 16-bit mono)	Prints format warning but still tries to play (may sound abnormal)
Short press BOOT again during playback (change track/replay)	Not supported in current version: <code>wav_player_play_file()</code> returns false, request ignored
Hold BOOT button then power on	GPIO0 is a strapping pin, may cause chip to enter download mode instead of normal boot

8. Common Issues

Symptom	Cause	Solution
No sound output	Amplifier not enabled / ES8311 not initialized	Check if GPIO7 is high
I2C scan finds no device	ES8311 module not soldered / wiring error	Check I2C pin connections (SCL=42, SDA=41)
Playback stutters	DMA buffer insufficient	Increase <code>dma_desc_num</code> or <code>dma_frame_num</code>
Abnormal playback speed	Sample rate mismatch	Ensure WAV file is 16kHz, matches code <code>EXAMPLE_SAMPLE_RATE</code>
Noise/pop during playback	Improper amplifier enable timing / I2S mute preamble not sent	Ensure amplifier enables after I2S outputs silence; send mute preamble (~25ms) before playback start

Symptom	Cause	Solution
SPIFFS has no files	Files not burned to SPIFFS partition	Use files from <code>spiffs_selected/</code> , confirm <code>partition table</code> has storage` partition
WS2812 not lit	RMT GPIO configuration error	Check GPIO48 connection, confirm LED power supply normal
Button unresponsive	GPIO0 occupied by other peripherals	GPIO0 is a strapping pin, ensure it's not pulled low at power-on
Only supports 16-bit mono	Code expects 1ch/16bit	Re-encode WAV file to mono 16-bit